

Risc-V Vector Extensions + FPU & MLU

Slide set 2

December 1, 2025

Outline

FPU & MLU

Risc-V Vector Extensions

Two types of FPU have been implemented:

- ▶ Indirect operations (Currently being used) - Converts the inputs into the corresponding real number, performs the operation based on the opcode and then converts the result back into the floating-point value (IEEE-754 32-bit).
- ▶ Direct operations - Operates on the individual bits

Testbench simulation:

fp_tb	0							
opcode_tb[2:0]	7	7	0	1	2	3		
operand_1_tb[31:0]	00000000	00000000	40600000	41200000	3fc00000	3f800000		
operand_2_tb[31:0]	00000000	00000000	40100000	40800000	c0000000	40000000		
output_tb[31:0]	00000000	00000000	40b80000	40c00000	c0400000	3f800000		

5	6	7	0	1	2	0
40b00000	0000002a	c0e80000	00000000		7f7fffff	ff7fffff
40b00000	00000000				7f7fffff	ff7fffff
00000001	36a80000	fc600000	00000000			

MLU

Implemented an machine learning unit (MLU) that currently performs mac and leaky relu (more instructions will be added, still under review). The operands can be floating-point values or integers (indicated by fp bit).

Testbench simulation:

[illegible]

Outline

FPU & MLU

Risc-V Vector Extensions

Vector Register Table

A new register table containing 32 vector registers will be created. Each vector register is of size 128 or 256, packing 4 or 8 32 bit values respectively. Elements are packed in increasing element index order into the bits of the register.

It is part of the instruction decode stage and uses the same bits of the instruction as the normal register.

Not implementing register grouping (LMUL).

Instructions

So far, the instructions that have been implemented can be duplicated for the vector extensions. The alu, fpu and mlu can be initialized multiple times to perform operations on each element (4 or 8) in a vector register.

The instruction is formatted as follows:

$$[\text{funct6}][\text{vm}][\text{vs2}][\text{vs1/rs1/imm}][\text{fmt}][\text{vd/rd}][1010111]$$

The 3-bit fmt field (bits 14:12) selects the variant:

- ▶ $\text{fmt} = 000$: OPIVV
- ▶ $\text{fmt} = 001$: OPFVV
- ▶ $\text{fmt} = 011$: OPIVI
- ▶ $\text{fmt} = 100$: OPIVX
- ▶ $\text{fmt} = 101$: OPFVF

Mask (vm)

The vm bit encodes masked vs unmasked variants. Mask registers themselves are vector registers (each bit i in a mask register corresponds to element i). Exact masked behavior and which mask register is used are defined per-instruction.

Convention: mask bit = 1 means the element is active (operation updates that element); mask bit = 0 means the element is inactive (masked-off).

To be implemented??

Load/Store

Options:

- ▶ Load/store for each element in a vector register.
- ▶ ?? (Need ideas)